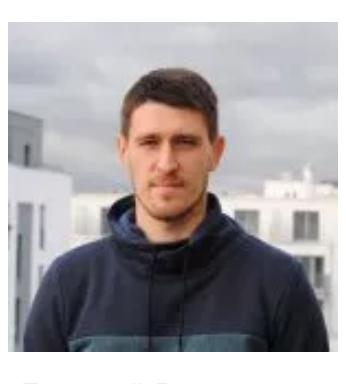
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Dagger 2
сделайте код чище
курс от автора сайта StartAndroid
MADE WITH MYBANNERMAKER.COM

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid
15 084 подписчика
Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 15. Hilt. Под капотом

20 августа 2021

В этом уроке разбираемся, что именно делает Hilt под капотом.

Hilt помогает нам в использовании даггера. От нас требуется только помечать специальными Hilt-аннотациями наши Android сущности (Application, Activity, сервисы, фрагменты, View, ViewModel), в которые мы хотим инджектиить объекты. А Hilt возьмет на себя создание иерархии требуемых даггер-компонентов и scope аннотаций, и спрячет их (а также выполнение инджекта) под капот.

Чтобы лучше понять, какую именно работу делает Hilt, давайте мы сами ее сделаем за него. В качестве примера возьмем простое приложение с двумя Activity: OrderActivity и UserActivity. А также в OrderActivity будем использовать фрагмент OrderFragment.

Чтобы пример получился максимально простым и понятным, мы пока не будем использовать модули и scope. О них поговорим позже, в отдельных уроках.

Важное замечание! Некоторым классам, которые мы создаем, будем добавлять слово Hilt в название. Таким образом мы пометим классы, которые Hilt создал бы за нас, если бы мы его использовали. Так будет проще понять, в чем именно состоит его работа.

Application

Начинаем с Application класса

```kotlin
class App : Application() {

}
```

Создаем App класс и не забываем добавить его в манифест.

Если мы хотим, чтобы даггер инджектил объекты в наш App класс, нам нужен компонент, который будет это делать. Давайте его создадим.

```kotlin
@Component
interface HiltAppComponent {

    fun injectApp(app: App)

}
```

Компонент с одним методом, который инджектит объекты в App. Далее будем называть его AppComponent.

Мы будем его использовать не напрямую в App классе, а создадим специальный базовый класс HiltApp, куда вынесем логику создания компонента и инджекта. А потом наш App наследуем от него.

Класс HiltApp:

```kotlin
open class HiltApp: Application() {

    val appComponent = DaggerHiltAppComponent.create()

    override fun onCreate() {
        super.onCreate()
        appComponent.injectApp(this as App)
    }

}
```

Все стандартно. Создаем компонент и вызываем его инджект метод в onCreate. Компонент проверит, есть ли в App переменные, помеченные аннотацией @Inject и поместит в них соответствующие объекты.

Теперь наследуем наш App от этого класса:

```kotlin
class App : HiltApp() {

}
```

В итоге у нас есть App класс, в который под капотом будут инджектиться объекты. Вся даггер магия ушла в классы HiltAppComponent и HiltApp. От нас требуется только добавлять Inject аннотации тем переменным, которые мы хотим получать от даггера.

Давайте создадим какой-нибудь класс для проверки того, что подкапотный инджект работает. Например, DatabaseHelper:

```kotlin
class DatabaseHelper @Inject constructor() {

}
```

Добавляем в класс App переменную databaseHelper с аннотацией Inject

```kotlin
class App : HiltApp() {

    @Inject
    lateinit var databaseHelper: DatabaseHelper

    override fun onCreate() {
        super.onCreate()
        Log.d(TAG, "databaseHelper = $databaseHelper")
    }

}
```

В методе onCreate после вызова super.onCreate (в котором и происходит инджект) мы получим готовый объект. И нам уже не надо думать о компонентах и инджекте.

Activity

Переходим к Activty. Сразу скажу, что здесь будет похожая схема. Мы будем создавать Hilt-сабкомпонент и Hilt-базовый класс для этого Activity. Но сабкомпонент будет работать немного по другой схеме, чем мы привыкли.

Создаем OrderActivity:

```kotlin
class OrderActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_order)
    }

}
```

Создаем для него сабкомпонент:

```kotlin
@Subcomponent
interface HiltActivityComponent {

    fun injectOrderActivity(activity: OrderActivity)

}
```

Обратите внимание на название сабкомпонента. Оно не содержит слова Order. Потому что этот сабкомпонент создается не под OrderActivity. Он будет универсальным и общим для всех Activity. В нем будут inject методы для каждого Activity. И каждое Activity будет получать свой экземпляр этого сабкомпонента и работать с ним.

Этот сабкомпонент мы будем называть ActivityComponent.

Получать его мы будем, как и в прошлых примерах, от AppComponent. Добавляем соответствующий get метод в HiltAppComponent:

```kotlin
@Component
interface HiltAppComponent {

    fun injectApp(app: App)

    fun getActivityComponent(): HiltActivityComponent

}
```

Теперь надо в OrderActivity получить сабкомпонент и выполнить инджект. Эту логику мы снова выносим в отдельный класс, который потом станет родителем OrderActivity.

Класс HiltOrderActivity:

```kotlin
open class HiltOrderActivity: AppCompatActivity() {

    lateinit var activityComponent: HiltActivityComponent

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        activityComponent = (application as App).appComponent.getActivityComponent()
        activityComponent.injectOrderActivity(this as OrderActivity)
    }

}
```

В onCreate берем AppComponent из App, просим у него создать нам ActivityComponent и вызываем инджект метод.

Наследуем наше Activity от этого класса:

```kotlin
class OrderActivity: HiltOrderActivity()
```

В итоге у нас есть OrderActivity класс, в который под капотом будут инджектиться объекты. А весь даггер-код спрятан в классах HiltActivityComponent и HiltOrderActivity.

Создадим класс для проверки инджекта, например OrderRepository:

```kotlin
class OrderRepository @Inject constructor() {

}
```

Добавим переменную OrderRepository в наш OrderActivity:

```kotlin
class OrderActivity : HiltOrderActivity() {

    @Inject
    lateinit var repository: OrderRepository

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_order)
        Log.d(TAG, "repository = $repository")
    }

}
```

В super.onCreate будет выполнен инджект, и мы получим репозиторий. А нам самим уже не надо возиться с компонентами и вызовом инджект или get методов. Все работает под капотом, в Hilt классах.

Второе Activity

Как мы уже выяснили чуть ранее, Hilt создает один общий сабкомпонент для всех Activity. Давайте посмотрим, что происходит при добавлении нового Activity.

Создадим UserActivity:

```kotlin
class UserActivity: AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_user)
    }

}
```

Если мы хотим в нем получать объекты от даггера, нам надо научить сабкомпонент делать это.

Для этого добавляем инджект метод в уже существующий ActivityComponent:

```kotlin
@Subcomponent
interface HiltActivityComponent {

    fun injectOrderActivity(activity: OrderActivity)
    fun injectUserActivity(activity: UserActivity)

}
```

Теперь сабкомпонент умеет инджектить объекты в оба наших Activity.

Создаем Hilt класс, в который спрячем весь даггер-код:

```kotlin
open class HiltUserActivity: AppCompatActivity() {

    lateinit var activityComponent: HiltActivityComponent

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        activityComponent = (application as App).appComponent.getActivityComponent()
        activityComponent.injectUserActivity(this as UserActivity)
    }

}
```

Делаем ровно то же, что мы делали в Hilt классе для OrderActivity. Берем AppComponent, просим его создать нам ActivityComponent и вызываем инджект метод сабкомпонента, чтобы даггер наполнил UserActivity нужными нам объектами.

Наследуем от этого класса наше UserActivity:

```kotlin
class UserActivity: HiltUserActivity()
```

Все готово. Теперь в это Activity даггер под капотом будет инджектить объекты.

Создадим репозиторий для проверки инджекта:

```kotlin
class UserRepository @Inject constructor() {

}
```

Добавляем переменную в Activity:

```kotlin
class UserActivity: HiltUserActivity() {

    @Inject
    lateinit var repository: UserRepository

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_user)
        Log.d(TAG, "repository = $repository")
    }

}
```

В super.onCreate будет выполнен инджект.

Каждое следующее Activity добавляется по этой же схеме. Под него создается инджект метод в сабкомпоненте и базовый Hilt класс, в котором спрятан даггер код.

Обсудим один важный момент. В Hilt классах, которые мы создаем для Activity, мы просим компонент AppComponent создать нам сабкомпонент ActivityComponent, вызывая метод getActivityComponent. Как мы знаем из урока про сабкомпоненты, такой метод при каждом вызове создает новый экземпляр сабкомпонента. В итоге каждое Activity получает свой собственный экземпляр сабкомпонента.

При этом сабкомпонент у нас универсальный. Он умеет инджектить объекты во все Activity приложения. Получается, что каждое Activity содержит свой экземпляр сабкомпонента, который умеет работать со всеми Activity. Не очень красиво с точки зрения SOLID принципов, но это плата за упрощение нашего кода.

В итоге каждое Activity просто вызывает свой инджект метод в сабкомпоненте, а остальные инджект методы игнорирует.

Фрагмент

Давайте добавим фрагмент. Реализация будет очень похожей на то, что мы делали для Activity.

Создаем фрагмент, который будем использовать в OrderActivity:

```kotlin
class OrderFragment : Fragment() {

    override fun onCreateView(inflater: LayoutInflater,
                              container: ViewGroup?,
                              savedInstanceState: Bundle?): View? {
        return inflater.inflate(R.layout.fragment_order, container, false)
    }

}
```

Создаем сабкомпонент, который будет инджектить объекты в этот фрагмент:

```kotlin
@Subcomponent
interface HiltFragmentComponent {

    fun injectOrderFragment(fragment: OrderFragment)

}
```

Этот сабкомпонент будет общим для всех фрагментов, аналогично тому, как ActivityComponent является общим для всех Activity. Далее мы будем называть его FragmentComponent.

Его родителем будет ActivityComponent. Добавим ему get метод для создания FragmentComponent:

```kotlin
@Subcomponent
interface HiltActivityComponent {

    fun injectOrderActivity(activity: OrderActivity)
    fun injectUserActivity(activity: UserActivity)

    fun getFragmentComponent(): HiltFragmentComponent

}
```

Далее для фрагмента создаем Hilt класс, куда вынесем всю работу с даггером:

```kotlin
open class HiltOrderFragment: Fragment() {

    lateinit var fragmentComponent: HiltFragmentComponent

    override fun onAttach(context: Context) {
        super.onAttach(context)
        fragmentComponent = (context as OrderActivity).activityComponent.getFragmentComponent(
        fragmentComponent.injectOrderFragment(this as OrderFragment)
    }

}
```

В методе onAttach мы у Activity берем ActivityComponent, просим его создать нам FragmentComponent и вызываем его инджект метод для наполнения фрагмента необходимыми нам объектами.

Обратите внимание, что в этом коде есть привязка к конкретному Activity - OrderActivity. Это не очень правильно. Мы не сможем использовать этот фрагмент в другом Activity. Я сделал так для упрощения примера.

*Чтобы это исправить, можно создать интерфейс с методом, который возвращает ActivityComponent. Этот интерфейс надо будет реализовать во всех Hilt*Activity классах. А в коде Hilt фрагментов мы просто будем приводить context к этому интерфейсу и получать от него ActivityComponent. Указывать конкретное Activity уже не придется.*

Остается только наследоваться от Hilt класса:

```kotlin
class OrderFragment: HiltOrderFragment()
```

И мы получим фрагмент с подкапотным инджектом.

Добавим в него переменную репозитория:

```kotlin
class OrderFragment: HiltOrderFragment() {

    @Inject
    lateinit var orderRepository: OrderRepository

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "fragment orderRepository = $orderRepository")
    }

    override fun onCreateView(inflater: LayoutInflater,
                              container: ViewGroup?,
                              savedInstanceState: Bundle?): View? {
        return inflater.inflate(R.layout.fragment_order, container, false)
    }

}
```

И в onCreate он нам уже доступен. На самом деле даже чуть раньше - в onAttach.

Hilt

Давайте еще раз быстро пробежимся по Hilt* классам и интерфейсам, которые мы создали.

App

HiltAppComponent (он же AppComponent) - даггер компонент для инджекта объектов в App класс. Кроме этого он создает ActivityComponent.

HiltApp - super класс для App. Выносит из App класса код получения компонента HiltAppComponent и вызова инджект метода.

Activity

HiltActivityComponent (он же ActivityComponent) - даггер сабкомпонент для инджекта объектов в Activity классы. Является общим для всех Activity. Кроме этого он создает FragmentComponent.

Hilt*Activity - отдельный super класс для каждого Activity. Выносит из Activity класса код получения сабкомпонента HiltActivityComponent и вызова инджект метода.

Фрагмент

HiltFragmentComponent (он же FragmentComponent) - даггер сабкомпонент для инджекта объектов в классы фрагментов. Является общим для всех фрагментов.

Hilt*Fragment - отдельный super класс для каждого фрагмента. Выносит из класса фрагмента код получения сабкомпонента HiltFragmentComponent и вызова инджект метода.

Примерно то же самое делает Hilt. Он создает для нас иерархию даггер компонентов. А также он создает супер-классы для наших Android сущностей (App, Activity, фрагмент), куда выносится весь даггер-код.

На следующем уроке мы сделаем такой же пример, но на этот раз все свои классы Hilt создаст сам.

Назад Вперёд

Комментарии

# **RE: Урок 15. Hilt. Под капотом** — **Михаил** 20.08.2021 19:37
Урррa, Hilt !!!!
Ответить

# **RE: Урок 15. Hilt. Под капотом** — **Сергей** 21.08.2021 12:19
Спасибо за отличный урок.

В своё время пытался делать @Inject в свои activity с помощью базового activity. У меня ничего не вышло.
Писал базовый класс активити BaseActivity. В нем писал appComponent.injectActivity(this), и сталкивался с ошибкой в ChildActivity, что Dagger не инжектит поля ChildActivity. Что вообще то логично - мы передаём в компонент для инджекта BaseActivity у которого просто нет полей ChildActivity.

Оказывается для того чтобы всё заработало нужно было сделать явное приведение в BaseActivity
appComponent.injectActivity(this as ChildActivity)
Ответить

# **Приведение as** — **Евгений** 12.09.2022 15:31
А мне так и не ясен этот момент - как можно родителя представить child?
Ответить

# **RE: Приведение as** — **Dmitry Vinogradov** 20.09.2022 19:07
Ну фактически этот код, хоть и написан в родителе, но выполняется в child объекте. Поэтому this в этом случае будет именно child
Ответить

# **As** — **Евгений** 23.09.2022 20:02
Спасибо, понял. А я уж думал опять магия kotlin мать и ё)
Ответить

# **As** — **Евгений** 24.09.2022 00:17
Хотя тогда такой вопрос - почему просто this нельзя оставить? Нам же не нужно здесь обращение к членам child, как в предыдущей строке с App.
Сорри, что не по теме, но просто интересно
Ответить

# **RE: As** — **Dmitry Vinogradov** 30.09.2022 14:52
Потому что мы вызываем метод
fun injectOrderActivity(activity: OrderActivity)
Он ждет от нас OrderActivity.

Разница в том, что когда в классе HiltUserActivity мы пишем this, то компилятор считает, что это объект HiltUserActivity, а не OrderActivity.

А уже во время выполнения в этом this будет находиться OrderActivity.
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.